

/*Reto 2*/

create database trazacafe;

/* En PostgreSQL no se usa 'use trazacafe;'. Se conecta directamente a la base de datos trazacafe */

```
create table finca(  
  idFinca serial primary key not null,  
  nombreFinca varchar (100) not null,  
  caficultorFinca varchar (50) not null,  
  municipioFinca varchar (50) not null,  
  departamentoFinca varchar (50) not null,  
  altitudMetros double precision not null  
);
```

```
create table cliente(  
  idCliente serial primary key not null,  
  nombreCliente varchar (100) not null  
);
```

```
create table pedido(  
  idPedido serial primary key not null,  
  fechaPedido date not null default  
    (current_date),  
  estadoPedido varchar (50) not null default  
    'Pendiente',  
  idClienteFK int not null,  
  constraint fkidPedidoCliente  
    foreign key (idClienteFK)
```

```

        references cliente(idCliente)

);

create table lote(
idLote serial primary key not null,
codigoLote varchar (50) not null,
variedadLote varchar (20) check (variedadLote in ('Castillo','Caturra','Geisha','Bourbon')) not
null,
proceso varchar (20) check (proceso in ('lavado','honey','natural')) not null,
fechaCosecha date not null,
kiloLote decimal (10,2) not null,
idFincaFK int not null,
constraint fkidFincaLote
        foreign key (idFincaFK)
        references finca(idFinca)
);

```

```

create table tostion (
idTostion serial primary key not null,
fechaTostion date not null,
kilosEntrada decimal (10,2) not null,
kilosSalida decimal (10,2) not null,
perfilTostion varchar (20) check (perfilTostion in ('claro','medio','oscuro')) not null,
idLoteFK int not null,
constraint fkidLoteTostion
        foreign key (idLoteFK)
        references lote(idLote)
);

```

```

create table catacion (
idCatacion serial primary key not null,
catador varchar (100) not null,
puntajeSCA decimal (5,2) not null,
fechaCatacion date not null,
idLoteFK int not null,
constraint fkidLoteCatacion
    foreign key (idLoteFK)
    references lote(idLote)
);

```

```

create table LineaPedido(
idLinea serial primary key not null,
kilosLP decimal (10,2) not null,
precioKG decimal (10,2) not null,
idPedidoFK int not null,
idTostionFK int not null,
constraint fkidPedidoLP
    foreign key (idPedidoFK)
    references pedido(idPedido),
constraint fkidTostionLP
    foreign key (idTostionFK)
    references tostion(idTostion)
);

```

/* En PostgreSQL equivale a: \d finca */

/*Reflexión reto 2: : ¿qué pasaría si intentas crear primero la tabla de lotes y después la de fincas? ¿Por qué?

Reflexión: Si intento crear primero la tabla

"lote" y después "finca",

va a haber un error porque "lote" tiene una llave

foránea que referencia a "fincas".

Entonces "finca" debe existir antes para que se

pueda establecer una relación entre ambas tablas,

por lo que el orden de creación debe ir en herencia*/

/*Reto 3*/

```
alter table finca
```

```
add constraint chk_finca_altitud check
```

```
(altitudMetros between 800 and 2500);
```

```
alter table catacion
```

```
add constraint chk_catacion_puntaje check
```

```
(puntajeSCA between 0 and 100);
```

```
alter table toston
```

```
add constraint chk_toston_kilos check
```

```
(kilosSalida <= kilosEntrada);
```

```
alter table lote
```

```
add constraint uq_lote_codigo unique
```

```
(codigoLote);
```

```
alter table pedido
```

```
add constraint chk_pedido_estado  
check (estadoPedido in  
( 'pendiente','confirmado','enviado','entregado','cancelado'));
```

```
alter table lote drop constraint fkidFincaLote;  
alter table lote  
    add constraint fkidFincaLote  
    foreign key (idFincaFK)  
    references finca(idFinca) on delete restrict;
```

```
alter table catacion drop constraint fkidLoteCatacion;  
alter table catacion  
    add constraint fkidLoteCatacion  
    foreign key (idLoteFK)  
    references lote(idLote) on delete cascade;
```

```
alter table pedido drop constraint fkidPedidoCliente;  
alter table pedido  
    add constraint fkidPedidoCliente  
    foreign key (idClienteFK)  
    references cliente(idCliente) on delete restrict;
```

```
alter table LineaPedido drop constraint fkidPedidoLP;  
alter table LineaPedido  
    add constraint fkidPedidoLP  
    foreign key (idPedidoFK)  
    references pedido(idPedido) on delete cascade;
```

```
alter table LineaPedido drop constraint fkidTostionLP;
```

```
alter table LineaPedido
    add constraint fkidTostionLP
    foreign key (idTostionFK)
    references tostion(idTostion) on delete restrict;
```

```
insert into finca (nombreFinca,
    caficultorFinca, municipioFinca, departamentoFinca, altitudMetros)
values ('Finca Prueba Altitud', 'N/A',
    'N/A', 'N/A', 3000);
```

/*SQL Error [23514]: ERROR: el valor para la columna «altitudmetros» viola la restricción de chequeo «chk_finca_altitud»*/

```
insert into finca (nombreFinca,
    caficultorFinca, municipioFinca, departamentoFinca, altitudMetros)
values ('Finca Prueba Base', 'N/A', 'N/A', 'N/A',
    1500);
```

```
insert into lote (codigoLote, variedadLote,
    proceso, fechaCosecha, kiloLote, idFincaFK)
values ('TEST-0000', 'Castillo', 'lavado',
    '2026-01-01', 100.00, curval('finca_idFinca_seq'));
```

```
insert into catacion (catador, puntajeSCA,
    fechaCatacion, idLoteFK)
values ('Catador Prueba', 105.00,
    '2026-01-02', (select idLote from lote where codigoLote =
    'TEST-0000'));
```

/*SQL Error [23514]: ERROR: el valor para la columna «puntajesca» viola la restricción de chequeo «chk_catacion_puntaje»*/

```
insert into lote (codigoLote,
variedadLote, proceso, fechaCosecha, kiloLote, idFincaFK)
values ('TEST-0000', 'Caturra', 'honey',
'2026-01-05', 50.00,
(select
idFinca from finca where nombreFinca = 'Finca Prueba Base'));
/*SQL Error [23505]: ERROR: llave duplicada viola restricción de unicidad «uq_lote_codigo»*/
```

```
insert into tostion (fechaTostion,
kilosEntrada, kilosSalida, perfilTostion, idLoteFK)
values ('2026-01-10', 50.00, 60.00,
'medio', (select idLote from lote where codigoLote =
'TEST-0000'));
/*SQL Error [23514]: ERROR: el valor viola la restricción de chequeo «chk_tostion_kilos»*/
```

```
insert into cliente (nombreCliente) values
('Cliente Prueba');
insert into pedido (estadoPedido,
idClienteFK)
values ('enviado_ya', (select
idCliente from cliente where nombreCliente = 'Cliente Prueba'));
/*SQL Error [23514]: ERROR: el valor viola la restricción de chequeo «chk_pedido_estado»*/
```

```
delete from lote where codigoLote =
'TEST-0000';
delete from finca where nombreFinca =
'Finca Prueba Base';
delete from cliente where nombreCliente
```

```
= 'Cliente Prueba';
```

```
alter sequence finca_idFinca_seq restart with 1;
```

```
alter sequence cliente_idCliente_seq restart with 1;
```

```
alter sequence lote_idLote_seq restart with 1;
```

```
alter sequence catacion_idCatacion_seq restart with 1;
```

```
alter sequence tostion_idTostion_seq restart with 1;
```

```
alter sequence pedido_idPedido_seq restart with 1;
```

/*Reflexión reto 3: ¿qué regla del caso no se puede expresar con un CHECK y por qué?

Reflexión: la regla "un lote puede catarse más de una vez, por catadores distintos" no se puede escribir como CHECK, porque un CHECK solo evalúa los valores de una misma fila contra sí misma; no puede contar ni comparar filas de otra tabla. Esa regla es de cardinalidad de 1 a muchos entre lote - catacion y ya queda solucionado con la creación de las llaves foránea/primaria, no con una restricción de dominio.*/*

/*Reto 4*/

```
alter table cliente
```

```
add column paisCliente varchar (60) not null default 'Colombia';
```

```
alter table tostion
```

```
add column huellaCarbonoKG decimal
(10,2) null;
alter table tostion
add constraint chk_tostion_huella check
(huellaCarbonoKG is null or huellaCarbonoKG >= 0);
```

```
create table certificacion(
idCertificacion serial primary key not null,
nombreCertificacion varchar (60) not null unique
);
```

```
create table fincaCertificacion(
idFincaFK int not null,
idCertificacionFK int not null,
primary key (idFincaFK, idCertificacionFK),
constraint fkidFincaFC
    foreign key (idFincaFK)
    references
finca(idFinca) on delete restrict,
constraint fkidCertificacionFC
    foreign key (idCertificacionFK)
    references
certificacion(idCertificacion) on delete restrict
);
```

```
alter table cliente
rename column nombreCliente to razonSocialCliente;
```

/*Reflexión reto 4: ¿por qué en producción es peligroso
borrar y recrear una tabla en lugar
de alterarla?

Reflexión: borrar y recrear una tabla en producción
significa que se pierden los datos que ya tenía,
quitar las llaves foráneas de las demás tablas que la
referencian (lote, pedido, etc.),
dejar el servicio sin esa tabla mientras se recrea, y dejar
sin utilidad los índices, vistas o
permisos que dependían de ella. ALTER TABLE cambia la
estructura sin quitar los datos ni
las relaciones existentes.*/

/*Reto 5*/

insert into finca (nombreFinca,
caficultorFinca, municipioFinca, departamentoFinca, altitudMetros) values
('La Esperanza', 'Rodrigo Muñoz', 'Pitalito', 'Huila',
1750),
('El Progreso', 'Sandra Trujillo', 'Acevedo', 'Huila',
1900),
('El Mirador', 'Marta Ortega', 'Buesaco', 'Nariño', 1980),
('Los Naranjos', 'Jorge Cadavid', 'Jardín', 'Antioquia',
1600);

insert into lote (codigoLote, variedadLote,

proceso, fechaCosecha, kiloLote, idFincaFK) values
(('HUI-2026-001', 'Castillo', 'lavado', '2026-03-01', 620.00,
1),
(('HUI-2026-002', 'Caturra', 'honey', '2026-03-10', 480.00,
1),
(('HUI-2026-003', 'Castillo', 'natural', '2026-02-20',
550.00, 2),
(('HUI-2026-004', 'Geisha', 'lavado', '2026-04-05', 210.00,
2),
(('NAR-2026-005', 'Bourbon', 'lavado', '2026-01-15', 600.00,
3),
(('NAR-2026-006', 'Caturra', 'honey', '2026-02-28', 430.00,
3),
(('ANT-2026-007', 'Castillo', 'lavado', '2026-03-18', 700.00,
4),
(('ANT-2026-008', 'Geisha', 'natural', '2026-04-22', 190.00,
4),
(('TEST-2026-999', 'Castillo', 'lavado', '2026-01-05', 5.00,
1);

insert into catacion (catador, puntajeSCA,
fechaCatacion, idLoteFK) values
(('Laura Peña', 84.50, '2026-03-15', 1),
(('Iván Cárdenas', 85.25, '2026-03-20', 1),
(('Laura Peña', 82.00, '2026-03-25', 2),
(('Iván Cárdenas', 78.50, '2026-03-01', 3),
(('Laura Peña', 89.75, '2026-04-10', 4),
(('Iván Cárdenas', 86.00, '2026-01-25', 5),
(('Laura Peña', 83.25, '2026-03-05', 6),

('Iván Cárdenas', 80.50, '2026-03-25', 7),
('Laura Peña', 81.50, '2026-03-28', 7),
('Iván Cárdenas', 90.00, '2026-04-29', 8),
('QA Interno', 70.00, '2026-01-06', 9);

insert into tostion (fechaTostion, kilosEntrada,
kilosSalida, perfilTostion, idLoteFK) values
('2026-04-01', 600.00, 510.00, 'medio', 1),
('2026-04-20', 200.00, 168.00, 'claro', 4),
('2026-02-01', 580.00, 490.00, 'medio', 5),
('2026-04-01', 680.00, 578.00, 'oscuro', 7),
('2026-05-02', 180.00, 150.00, 'claro', 8);

insert into cliente (razonSocialCliente,
paisCliente) values
('Tostadora Andina S.A.S.', 'Colombia'),
('BerlinBohnen Rösterei', 'Alemania'),
('Cafés del Pacífico Export', 'Colombia');

insert into pedido (fechaPedido, estadoPedido,
idClienteFK) values
('2026-04-15', 'pendiente', 2),
('2026-04-18', 'confirmado', 1),
('2026-04-20', 'pendiente', 3),
('2026-05-01', 'pendiente', 2);

insert into LineaPedido (kilosLP, precioKG,
idPedidoFK, idTostionFK) values
(100.00, 45000.00, 1, 1),

```
(50.00, 68000.00, 1, 2),  
(80.00, 42000.00, 2, 3),  
(120.00, 39000.00, 3, 4),  
(60.00, 71000.00, 3, 5),  
(40.00, 45500.00, 4, 1),  
(30.00, 42500.00, 4, 3);
```

```
insert into certificacion  
(nombreCertificacion) values  
('Orgánico'), ('Fair Trade');
```

```
insert into fincaCertificacion (idFincaFK,  
idCertificacionFK) values  
(1, 1), (1, 2), (3, 1), (4, 2);
```

```
create table loteEspecialidad (  
codigoLote varchar (50) primary key,  
puntajePromedio decimal (5,2) not null  
);
```

```
insert into loteEspecialidad (codigoLote,  
puntajePromedio)  
select l.codigoLote, avg(c.puntajeSCA)  
from lote l  
join catacion c on c.idLoteFK = l.idLote  
group by l.codigoLote  
having avg(c.puntajeSCA) >= 85;
```

```
select * from loteEspecialidad;
```

/*Reflexión reto 5: ¿lotes_especialidad se actualiza sola cuando llega una nueva catación?
¿Qué objeto usarías para que sí lo hiciera?

Reflexión: loteEspecialidad es una tabla con una copia del cálculo tomada en
INSERT SELECT. Si entra una catación nueva que cambia el promedio de un
lote, la tabla queda desactualizada hasta que se vuelva a correr la consulta. Para que
se mantuviera sincronizada sola reemplazaría la tabla por una VIEW que
siempre calcula el promedio en el momento de consultarla.*/

/*Reto 6*/

```
create table precioReferencia (  
variedadPR varchar (30) primary key,  
precioKG decimal (10,2) not null,  
actualizadoEn timestamp not null default current_timestamp  
);
```

```
insert into precioReferencia (variedadPR,  
precioKG) values  
('Castillo', 32000.00),  
('Caturra', 35500.00),  
('Geisha', 120000.00);
```

```
insert into precioReferencia (variedadPR,  
precioKG) values  
( 'Caturra', 36800.00),  
( 'Geisha', 118000.00),  
( 'Bourbon', 41000.00)  
on conflict (variedadPR) do update set  
    precioKG = excluded.precioKG,  
    actualizadoEn = now();
```

```
select count(*) as totalVariedades from  
precioReferencia;  
select * from precioReferencia;
```

/*Reflexión reto 6: ¿qué pasaría si variedad no fuera clave
primaria ni UNIQUE?

Reflexión: si no fuera PK ni UNIQUE en variedadPR, ON
DUPLICATE KEY UPDATE no tendría contra qué
índice comparar la fila nueva para detectar un
"duplicado". Sin esa restricción, cada INSERT agregaría una fila
nueva en vez de actualizar: se terminaría con varias filas
de "Caturra" con precios distintos
en lugar de una sola fila actualizada.*/

/*Reto 7*/

```
select count(*) as cataciones_a_corregir
```

```
from catacion
```

```
where catador = 'Iván Cárdenas';
```

```
update catacion
```

```
set puntajeSCA = greatest(puntajeSCA - 1.5, 0)
```

```
where catador = 'Iván Cárdenas';
```

```
select count(*) as lineas_a_descontar
```

```
from LineaPedido lp
```

```
join pedido p on p.idPedido = lp.idPedidoFK
```

```
join cliente c on c.idCliente = p.idClienteFK
```

```
where c.razonSocialCliente = 'BerlinBohnen Rösterei';
```

```
update LineaPedido lp
```

```
set precioKG = lp.precioKG * 0.90
```

```
from pedido p
```

```
join cliente c on c.idCliente = p.idClienteFK
```

```
where p.idPedido = lp.idPedidoFK and c.razonSocialCliente = 'BerlinBohnen Rösterei';
```

```
select * from LineaPedido;
```

/*Reflexión reto 7: ¿qué habría pasado si ejecutabas el

UPDATE del descuento dos veces?

Reflexión: el descuento se habría aplicado sobre el precio

ya descontado,

es decir, un 19% de descuento acumulado en vez del 10%

pactado.*/*

/*Reto 8*/

delete from finca where

idFinca = 1;

alter table finca add column

activaFinca smallint not null default 1;

update finca set activaFinca = 0 where

idFinca = 1;

select idFinca, nombreFinca, activaFinca from

finca;

delete from catacion c

using lote l

where l.idLote = c.idLoteFK and l.codigoLote = 'TEST-2026-999';

truncate table loteEspecialidad;

drop table loteEspecialidad;

/*Reflexión reto 8: explica en tus palabras la diferencia

entre DELETE, TRUNCATE y DROP.

¿Cuál es DML y cuáles DDL?

Reflexión: DELETE es DML: borra filas específicas, admite

WHERE, dispara triggers de fila.

TRUNCATE y DROP son DDL: TRUNCATE vacía la tabla completa de una sola vez (sin WHERE),
reinicia el AUTO_INCREMENT y en MySQL no se puede revertir;
DROP elimina la estructura
misma de la tabla, no solo sus filas.* /

/*Reto 9*/

```
alter table tostion add column  
kilosDisponibles decimal (10,2) null;
```

```
update tostion t  
set kilosDisponibles = t.kilosSalida - coalesce(  
    (select sum(lp.kilosLP) from LineaPedido lp where lp.idTostionFK =  
    t.idTostion), 0);
```

```
alter table tostion alter column  
kilosDisponibles set not null;  
alter table tostion add constraint  
chk_tostion_disponibles check (kilosDisponibles >= 0);
```

```
select idTostion, kilosSalida, kilosDisponibles from  
tostion;
```

```
begin;
```

```
insert into pedido (fechaPedido, estadoPedido,  
idClienteFK) values ('2026-05-10', 'pendiente', 1);
```

```
insert into LineaPedido (kilosLP, precioKG,  
idPedidoFK, idTostionFK) values (30.00, 68000.00, 5, 2);  
update tostion set kilosDisponibles =  
kilosDisponibles - 30.00 where idTostion = 2;
```

```
insert into LineaPedido (kilosLP, precioKG,  
idPedidoFK, idTostionFK) values (20.00, 71000.00, 5, 5);  
update tostion set kilosDisponibles =  
kilosDisponibles - 20.00 where idTostion = 5;
```

```
commit;
```

```
begin;
```

```
insert into pedido (fechaPedido, estadoPedido,  
idClienteFK) values ('2026-05-11', 'pendiente', 3);
```

```
insert into LineaPedido (kilosLP, precioKG,  
idPedidoFK, idTostionFK) values (5000.00, 39000.00, 6, 4);
```

```
update tostion set kilosDisponibles  
= kilosDisponibles - 5000.00 where idTostion = 4;
```

```
/*SQL Error [23514]: ERROR: el valor para la columna «kilosdisponibles» viola la restricción de  
chequeo «chk_tostion_disponibles»*/
```

```
rollback;
```

```
select * from pedido where idPedido =
```

6;

begin;

insert into pedido (fechaPedido, estadoPedido,
idClienteFK) values ('2026-05-12', 'pendiente', 2);

insert into LineaPedido (kilosLP, precioKG,
idPedidoFK, idTostionFK) values (10.00, 45000.00, 7, 1);
update tostion set kilosDisponibles =
kilosDisponibles - 10.00 where idTostion = 1;

savepoint antes_linea_2;

insert into LineaPedido (kilosLP, precioKG,
idPedidoFK, idTostionFK) values (15.00, 42000.00, 7, 3);
update tostion set kilosDisponibles = kilosDisponibles
- 15.00 where idTostion = 3;

rollback to savepoint antes_linea_2;

commit;

select * from pedido where idPedido =
7;
select * from LineaPedido where
idPedidoFK = 7;

```
begin;
```

```
create table prueba (id int);
```

```
rollback;
```

```
select * from prueba;
```

```
/*SQL Error [42P01]: ERROR: no existe la relación «prueba»*/
```

```
drop table if exists prueba;
```

```
/*Reflexión reto 9: ¿qué descubriste en el punto 5 y qué
```

```
riesgo implica para un script de
```

```
migración en MySQL?
```

Reflexión: en MySQL cualquier sentencia DDL (como CREATE TABLE) provoca un

COMMIT implícito de toda la transacción en curso, así que la tabla "prueba" siguió

existiendo después. Es decir, que un script que se mezcle

DDL y DML dentro de una misma

si algo falla después del DDL, el ROLLBACK no puede deshacer

los cambios de estructura que ya quedaron confirmados.*/*

```
/*Reto 10*/
```

```
drop table if exists prueba;
```

```
create or replace view
```

```
vTrazabilidad as
```

```

select
    lp.idPedidoFK as pedido,
    lp.idLinea as lineaPedido,
    c.razonSocialCliente as cliente,
    c.paisCliente as paisCliente,
    f.nombreFinca as finca,
    f.caficultorFinca as caficultor,
    f.municipioFinca as municipio,
    f.altitudMetros as altitud,
    l.variedadLote as variedad,
    l.proceso as proceso,
    (select avg(cat.puntajeSCA) from catacion cat where cat.idLoteFK =
l.idLote) as puntajePromedio,
    t.fechaTostion as fechaTostion,
    t.perfilTostion as perfil,
    (select string_agg(cert.nombreCertificacion, ' ' )
        from
fincaCertificacion fc
        join
certificacion cert on cert.idCertificacion = fc.idCertificacionFK
        where
fc.idFincaFK = f.idFinca) as certificaciones,
    concat(
        l.variedadLote,
        ', ' , l.proceso, ' de Finca ', f.nombreFinca, ', ', f.municipioFinca,
        ',
        (', f.altitudMetros, ' m). Puntaje ',
        round((select avg(cat.puntajeSCA) from catacion cat where cat.idLoteFK =
l.idLote), 2),

```

```

        '
    Tostado', t.perfilTostion, ' el ', t.fechaTostion, '
    ) as
    texto_qr
    from LineaPedido lp
    join pedido p on p.idPedido = lp.idPedidoFK
    join cliente c on c.idCliente = p.idClienteFK
    join tostion t on t.idTostion = lp.idTostionFK
    join lote l on l.idLote = t.idLoteFK
    join finca f on f.idFinca = l.idFincaFK;

```

```

select * from vTrazabilidad where
pedido = 1;

```

/*Reto creativo: se agregó la columna
 "certificaciones" porque al consumidor final que escanea
 el QR le interesa tanto la historia del café como saber si
 la finca cumple estándares que él valora al comprar.*/

/*Reflexión reto 10: ¿una vista guarda datos? ¿Qué ventaja
 tiene sobre copiar los datos a
 una tabla como hiciste en el reto 5?

Reflexión: una vista no guarda datos, es una consulta
 guardada que el motor vuelve a
 ejecutar cada vez que se consulta, así que siempre muestra
 el estado actual de las tablas
 base. Frente a copiar los datos a una tabla, la ventaja es

que nunca queda desactualizada

ni exige un proceso aparte para actualizarla, es decir, no

hay riesgo de inconsistencia entre la copia y el dato real.*/